BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

High-Level and Low-Level Languages

Name _____ **Class** _____ **Date** _____ **Time** _____ **Teacher** _____

Do Now - Four retrieval questions

1. Evaluate (A AND B) OR C for A=1, B=0, C=1.

2. What is denary 182 in unsigned 8-bit binary?
   A) 10101110
   B) 11010110
   C) 10110110
   D) 01101101

3. Add 00010111 and 00000110 as unsigned binary numbers. Give the eight-bit result.

4. (a) Logically shift unsigned eight-bit 00110001 one place left. Give the eight-bit result and denary value. (b) Logically shift unsigned eight-bit 00110001 one place right. Give the eight-bit result and denary value.

Today's objectives

Recognise high-level and low-level languages. Explain why programs need translation into machine code.

Key vocabulary - Read, cover, recall

High-level language
A programming language with readable instructions less tied to a particular CPU.

Low-level language
A language close to machine instructions, such as assembly language or machine code.

Machine code
Binary instructions the CPU can execute.

Portability
How easily a program can be moved to different hardware or systems.

BLACK PEN: Write one definition in your own words.

Do Now	I Do	We Do	You Do	Review & Improve
Recall previous learning	Read new knowledge	Practise with guidance	Work independently	Check my understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Learn and practise

New Learning - Key ideas

High-level languages

High-level code is designed to be readable: for example, score = score + 1. It is usually easier to develop, debug and maintain than processor-specific instructions.

Low-level languages

Machine code is made of binary instructions executed by a CPU. Assembly is a low-level form using short instruction mnemonics: you only need to recognise the difference, not learn an assembler.

Why translate?

Processors execute machine code. Source programs written in high-level languages need translation into executable instructions before or while running.

Choose for the task

A school application benefits from readable, maintainable, portable code. Direct processor control may justify a lower-level approach, but is harder to maintain and is processor-dependent.

I Do - Worked model

A school is developing a timetable application. Explain TWO benefits of using a high-level language. [4 marks]

- The application will be maintained and may be used on different computers. Choose benefits relevant to that context.
- High-level code is more readable [1], so staff can understand and change the program more quickly [1].
- High-level code is generally more portable [1], so moving to a different processor is likely to require less rewriting [1].
- Award a knowledge mark and a linked consequence for EACH explained benefit. Do not simply name two features.

We Do - Guided practice

1. A controller needs precise access to processor instructions. Explain ONE advantage and ONE disadvantage of using a low-level language. [4 marks]

---

2. Compare high-level with low-level code in terms of readability and portability. [4 marks]

---

3. A school wants an attendance application that teachers can edit and run on different computers. Recommend a language level and explain TWO reasons. [4 marks]

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Think and write

Code First - Predict > Write > Test > Improve

YOUR TASK: Write one OCR ERL line to print the character represented by ASCII code 66.

```
code = 66
```

You Do - Independent exam questions

Q2. A new colleague takes over a school reports program. Explain TWO advantages of the code having been written in a high-level language. [4 marks]

Q3. A computer is upgraded to a processor with a different instruction set. Explain TWO possible drawbacks of an existing low-level program. [4 marks]

Q4. A medical practice is creating an appointment-booking program that will need regular updates. Explain TWO advantages of using a high-level language. [4 marks]

Exit Ticket - True or False?

Machine code is made of processor instructions that the CPU executes.

TRUE [ ] **FALSE [ ]**

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & improve

GREEN PEN: Tick each correct point; correct mistakes.

Do Now - Model answers

1. 1: A AND B is 0, then 0 OR 1 is 1.

2. C - 10110110

3. 00011101; 23 + 6 = 29.

4. (a) 01100010 = 98. Fill the empty bit with 0. (b) 00011000 = 24. Fill with 0 and lose the old rightmost bit.

I Do - Explain the model

The application will be maintained and may be used on different computers. Choose benefits relevant to that context.

High-level code is more readable [1], so staff can understand and change the program more quickly [1].

High-level code is generally more portable [1], so moving to a different processor is likely to require less rewriting [1].

Award a knowledge mark and a linked consequence for EACH explained benefit. Do not simply name two features.

We Do - Model answers

1. Instructions can give precise processor control [1] useful for managing the device hardware [1]. They are processor-specific [1], so code may need rewriting for different hardware [1].

2. High-level programs are usually easier for people to read [1] and are more portable [1]. Low-level code corresponds closely to CPU instructions [1] and is usually processor-specific [1].

3. High-level [1]; the code is easier to read and maintain as rules change [1]; it is usually more portable [1], making it easier to move across processor families [1].

Code First - Answer

```
code = 66
print(CHR(code))
```
Expected output: B

You Do - Self-mark

Q2: High-level code is easier to read [1], helping the colleague understand existing instructions [1]. It is generally easier to maintain [1], making changes and fault correction quicker [1].

Q3: Low-level instructions are often specific to the original processor [1], so the new processor might not execute the code [1]. The program may need rewriting for the new instruction set [1], increasing development time and maintenance costs [1].

Q4: Readable code makes maintenance and correction easier [1], saving development time when appointment rules change [1]. High-level code is generally portable [1], reducing rewriting if the system is moved to other processor types [1].

GREEN PEN: Choose the hardest answer. Correct it on lined paper.